Rebuild Playbook - MCBT

Panduan eksekusi membangun aplikasi dari nol: database, backend, frontend, infrastruktur, verifikasi.

Dokumen ini adalah resep reproduksi lengkap aplikasi MCBT. Ikuti langkah secara berurutan; setiap step menyatakan apa yang sudah ada, apa yang dibangun, endpoint lengkap dengan contoh respons, dan kriteria selesai (acceptance criteria). Cukup detail untuk dieksekusi oleh manusia maupun AI tanpa bertanya lagi.

DAFTAR ISI

- 0. Overview & Konvensi Global
- 1. DATABASE - Skema Lengkap
- 2. FASE BACKEND (B1-B15)
- 3. FASE FRONTEND (F1-F12)
- 4. INFRASTRUKTUR
- 5. VERIFIKASI AKHIR

0. Overview & Konvensi Global

0.1 Tujuan aplikasi

Aplikasi ujian berbasis komputer (CBT) untuk sekolah dengan 3 role:

```
| Role | Hak akses ringkas |
| **admin** | Semua modul: master data (tahun ajaran, kelas, mapel, guru, siswa), roles, bank soal, ujian, nilai,
laporan, ekspor |
| **teacher** | CRUD bank soal & ujian **miliknya sendiri**, lihat semua bank (read-only milik orang lain),
menangani laporan pada ujiannya, menu siswa read-only |
| **student** | Ujian yang di-assign, mengerjakan (timer server), hasil & pembahasan sendiri, lapor soal |
```

0.2 Stack & versi

```
| Lapisan | Teknologi |
| Backend | Go 1.27, Gin, GORM, PostgreSQL 16, golang-jwt, bcrypt, excelize v2, go-pdf/fpdf, golang-migrate, slog
|
| Frontend | Vue 3 + TypeScript, Vite, Tailwind CSS v4, Pinia, Vue Router, Axios, Lucide, Vitest |
| Infra | Docker Compose: web (Nginx), api, postgres, redis, minio |
```

0.3 Prasyarat

- Go 1.27+, Node.js 20+, Docker (opsional), PostgreSQL 16 lokal
- Git, `make` (opsional)

0.4 Struktur folder final

```
.
|-- cmd/
|   |-- api/main.go          # entrypoint server
|   |-- migrate/main.go      # CLI migrasi (up/down/status)
|   +-- gendocs/main.go      # generator PDF dokumentasi (opsional)
|-- internal/
|   |-- config/config.go     # load .env / env vars
|   |-- database/            # koneksi GORM
|   |-- model/               # entitas GORM (1 file per domain)
|   |-- repository/          # akses data (1 file per domain)
|   |-- usecase/             # logika bisnis (+ interfaces.go, fakes_test.go)
|   |-- delivery/http/handler/
|   |-- server/              # router.go, middleware/, scope.go
|   +-- pkg/                 # apperror, response, jwt, password, storage, logger
|-- migrations/              # 000001..000021 (up/down SQL)
|-- frontend/
|   |-- src/{pages,components,services,stores,composables,router,lib,types}
|   |-- Dockerfile, nginx.conf
|-- docs/                    # openapi.yaml, PDF dokumentasi
|-- Dockerfile               # backend multi-stage
|-- docker-compose.yml
+-- .env.example
```

0.5 Konvensi global (WAJIB konsisten)

Envelope respons sukses:

```
{ "success": true, "message": "pesan", "data": { }, "meta": { "page": 1, "limit": 10, "total_items": 42,
"total_pages": 5 } }
```

Envelope respons error:

```
{ "success": false, "message": "Validasi gagal", "errors": { "field": "alasan" } }
```

- Kode HTTP: 200/201 sukses; 400 bad request; 401 unauthenticated; 403 role/scope; 404 not found; 409 duplikat/konflik; 422 validasi; 500 internal.
- Semua ID UUID v4; timestamp `timestamptz` UTC.
- Auth: JWT di cookie HttpOnly (`access_token` 15 menit, `refresh_token` 7 hari) + cookie `csrf_token` (bukan

HttpOnly).
- Semua metode tidak aman (POST/PUT/PATCH/DELETE) wajib header `X-CSRF-Token` = nilai cookie `csrf_token`.
- Password default user baru: `McBT@1234`; admin seed: `admin123` / `Admin@123`.
- List selalu paginated (`page`, `limit`, `search`), meta di `meta`.
- Soft-delete TIDAK dipakai (hard delete), kecuali disebutkan.

0.6 Urutan eksekusi & gate

```
| Fase | Gate sebelum lanjut |
| Database | `migrate up` sukses, seed admin bisa login via curl |
| Backend | `go build ./... && go test ./...` hijau; smoke test tiap modul via curl |
| Frontend | `npm run build` hijau; alur login -> dashboard jalan |
| Infra | `docker compose up` -> aplikasi jalan penuh di :5173 |
```

---

1. DATABASE - Skema Lengkap

Buat tool migrasi (`cmd/migrate/main.go`) memakai `golang-migrate` membaca folder `migrations/` dengan pola `0000NN_nama.up.sql` / `.down.sql`. Buat file migrasi berikut (ringkasan DDL inti - kolom `id UUID PK DEFAULT gen_random_uuid()`, `created_at`/`updated_at TIMESTAMPTZ NOT NULL DEFAULT now()` ada di semua tabel kecuali disebut lain):

0001 - identity

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(150) NOT NULL UNIQUE,
  name VARCHAR(100) NOT NULL,
  password_hash TEXT NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT true,
  token_version INT NOT NULL DEFAULT 1,
  last_login_at TIMESTAMPTZ
);
CREATE TABLE roles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(30) NOT NULL UNIQUE
);
CREATE TABLE user_roles (
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role_id UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
  PRIMARY KEY (user_id, role_id)
);
INSERT INTO roles (name) VALUES ('admin'), ('teacher'), ('student');
-- seed admin: password_hash = bcrypt('Admin@123'), username 'admin123'
```

0002-0003 - akademik

```
CREATE TABLE academic_years (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  year VARCHAR(9) NOT NULL,             -- "2026/2027"
  semester VARCHAR(10) NOT NULL,        -- ganjil|genap
  is_active BOOLEAN NOT NULL DEFAULT false,
  UNIQUE (year, semester)
);
CREATE TABLE subjects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code VARCHAR(20) NOT NULL UNIQUE,
  name VARCHAR(100) NOT NULL,
  description TEXT
);
CREATE TABLE classes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  academic_year_id UUID NOT NULL REFERENCES academic_years(id),
  name VARCHAR(50) NOT NULL,
  homeroom_teacher_id UUID REFERENCES teachers(id),  -- dibuat setelah teachers
  UNIQUE (academic_year_id, name)
);
```

0004 - guru & siswa (butuh users)

```
CREATE TABLE teachers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL UNIQUE REFERENCES users(id),
  nip VARCHAR(30) UNIQUE, phone VARCHAR(20), address VARCHAR(255)
);
```

```
CREATE TABLE students (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL UNIQUE REFERENCES users(id),
  class_id UUID REFERENCES classes(id),
  nis VARCHAR(30) NOT NULL UNIQUE,
  phone VARCHAR(20), address VARCHAR(255)
);
CREATE INDEX idx_students_class_id ON students(class_id);
```

0006 - token_version & username (jika belum ada di 0001, tambahkan di sini)

0010 - bank soal & soal

```
CREATE TABLE question_banks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  subject_id UUID NOT NULL REFERENCES subjects(id),
  academic_year_id UUID REFERENCES academic_years(id),
  created_by UUID REFERENCES users(id),          -- pemilik bank (data scope guru)
  code VARCHAR(50) NOT NULL UNIQUE,
  title VARCHAR(150) NOT NULL,
  description TEXT,
  status VARCHAR(20) NOT NULL DEFAULT 'draft'    -- draft|published|archived
);
CREATE TABLE questions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  question_bank_id UUID NOT NULL REFERENCES question_banks(id) ON DELETE CASCADE,
  question_type VARCHAR(20) NOT NULL,  -- multiple_choice|true_false|multiple_answer|essay|short_answer
  content TEXT NOT NULL,
  score_weight NUMERIC(6,2) NOT NULL DEFAULT 1.0,
  explanation TEXT,
  answer_keys TEXT,                    -- kunci esai/isian, dipisah newline
  media_id UUID, media_position VARCHAR(10) DEFAULT 'after',
  is_used BOOLEAN NOT NULL DEFAULT false
);
CREATE TABLE options (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  question_id UUID NOT NULL REFERENCES questions(id) ON DELETE CASCADE,
  label VARCHAR(5) NOT NULL,           -- A..E
  content TEXT NOT NULL,
  is_correct BOOLEAN NOT NULL DEFAULT false,
  position INT NOT NULL DEFAULT 0,
  media_id UUID
);
```

0012-0014 - ujian, section, jadwal, peserta

```
CREATE TABLE exams (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title VARCHAR(150) NOT NULL,
  description TEXT,
  subject_id UUID NOT NULL REFERENCES subjects(id),
  academic_year_id UUID REFERENCES academic_years(id),
  question_bank_id UUID REFERENCES question_banks(id),  -- opsional (konsep: bank lewat section)
  created_by UUID REFERENCES users(id),                 -- pemilik ujian (scope guru)
  status VARCHAR(20) NOT NULL DEFAULT 'draft',          -- draft|published|closed
  duration_minutes INT NOT NULL DEFAULT 60,
  max_attempts INT NOT NULL DEFAULT 1,
  passing_grade NUMERIC(5,2) NOT NULL DEFAULT 75,
  randomize_questions BOOLEAN NOT NULL DEFAULT false,
  randomize_options BOOLEAN NOT NULL DEFAULT false,
  allow_backtrack BOOLEAN NOT NULL DEFAULT true,
```

```
  auto_submit BOOLEAN NOT NULL DEFAULT true,
  show_result_immediately BOOLEAN NOT NULL DEFAULT false,
  negative_marking BOOLEAN NOT NULL DEFAULT false,
  negative_value NUMERIC(4,2) NOT NULL DEFAULT 0,
  token_enabled BOOLEAN NOT NULL DEFAULT false,
  results_published BOOLEAN NOT NULL DEFAULT false,
  allow_discussion BOOLEAN NOT NULL DEFAULT false,
  exam_token VARCHAR(10)
);
CREATE INDEX idx_exams_created_by ON exams(created_by);

CREATE TABLE exam_sections (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  exam_id UUID NOT NULL REFERENCES exams(id) ON DELETE CASCADE,
  name VARCHAR(100) NOT NULL,
  sequence INT NOT NULL
);
CREATE TABLE exam_section_questions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  section_id UUID NOT NULL REFERENCES exam_sections(id) ON DELETE CASCADE,
  question_id UUID NOT NULL REFERENCES questions(id),
  UNIQUE (section_id, question_id)
);
CREATE TABLE exam_schedules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  exam_id UUID NOT NULL REFERENCES exams(id) ON DELETE CASCADE,
  start_time TIMESTAMPTZ NOT NULL,
  end_time TIMESTAMPTZ NOT NULL,
  token VARCHAR(10) NOT NULL
);
CREATE TABLE exam_participants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  exam_id UUID NOT NULL REFERENCES exams(id) ON DELETE CASCADE,
  student_id UUID NOT NULL REFERENCES students(id) ON DELETE CASCADE,
  assigned_via VARCHAR(12) NOT NULL DEFAULT 'class',   -- class|individual
  UNIQUE (exam_id, student_id)
);
```

0015-0018 - attempt, jawaban, penilaian

```
CREATE TABLE exam_attempts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  exam_id UUID NOT NULL REFERENCES exams(id),
  student_id UUID NOT NULL REFERENCES students(id),
  attempt_no INT NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'in_progress',  -- in_progress|submitted|expired
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at TIMESTAMPTZ NOT NULL,
  submitted_at TIMESTAMPTZ,
  score NUMERIC(5,2)
);
CREATE INDEX idx_attempts_exam_student ON exam_attempts(exam_id, student_id);

CREATE TABLE exam_answers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  attempt_id UUID NOT NULL REFERENCES exam_attempts(id) ON DELETE CASCADE,
  question_id UUID NOT NULL REFERENCES questions(id),
  answer_value TEXT NOT NULL DEFAULT '',
  client_timestamp BIGINT NOT NULL DEFAULT 0,
  is_flagged BOOLEAN NOT NULL DEFAULT false,
  answered_at TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
  score NUMERIC(6,2), is_correct BOOLEAN,
  feedback TEXT, graded_at TIMESTAMPTZ, graded_via VARCHAR(10),  -- auto|manual
  UNIQUE (attempt_id, question_id)
);
```

0019-0021 - laporan, media

```
CREATE TABLE question_reports (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  attempt_id UUID NOT NULL REFERENCES exam_attempts(id) ON DELETE CASCADE,
  question_id UUID NOT NULL REFERENCES questions(id),
  student_id UUID NOT NULL REFERENCES students(id),
  reason TEXT NOT NULL,
  status VARCHAR(15) NOT NULL DEFAULT 'pending',      -- pending|reviewing|resolved|rejected
  resolution TEXT, resolved_by UUID REFERENCES users(id), resolved_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE media (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  key VARCHAR(200) NOT NULL,           -- object key di S3/MinIO
  filename VARCHAR(200) NOT NULL,
  content_type VARCHAR(100) NOT NULL,
  size_bytes BIGINT NOT NULL
);
ALTER TABLE questions   ADD COLUMN IF NOT EXISTS allow_discussion_refs TEXT; -- (tidak dipakai, contoh)
ALTER TABLE exams       ADD COLUMN IF NOT EXISTS allow_discussion BOOLEAN NOT NULL DEFAULT FALSE;
```

Seed

- 3 roles + user `admin123` (bcrypt `Admin@123`, role admin, is_active true).

Acceptance: `migrate up` sukses; `psql` menampilkan 20 tabel; login admin via endpoint auth (Bagian 2, B3) mengembalikan 200.

---

2. FASE BACKEND

B1 - Scaffold & paket dasar

Tujuan: proyek jalan, config terbaca, response/error konsisten.

File: `go.mod`, `cmd/api/main.go` (graceful shutdown), `internal/config/config.go` (baca `.env` lalu env vars: `APP_*, DB_*, JWT_*, COOKIE_*, S3_*, LOG_LEVEL`), `internal/database/database.go` (GORM Open + pool), `internal/pkg/logger` (slog JSON), `internal/pkg/response` (Success/SuccessWithMeta/Error), `internal/pkg/apperror` (struct Code/Message/Details + helper BadRequest/Unauthorized/Forbidden/NotFound/Unprocessable/Internal/From), `internal/server/middleware`: RequestID, RequestLogger, Recovery, ErrorHandler.

Acceptance: `GET /health` -> `200 {"success":true,"data":{"status":"UP"}}`.

B2 - Model & migrasi

Tujuan: entitas GORM 1:1 dengan skema Bagian 1.

File: `internal/model/*.go` (user, role, academic_year, class, subject, teacher, student, question_bank, question, media, exam, exam_section, exam_schedule, exam_participant, exam_attempt, exam_answer, question_report). Semua embed `BaseModel{ID, CreatedAt, UpdatedAt}`.

Acceptance: `go run ./cmd/migrate up` sukses; tabel sesuai Bagian 1.

B3 - Auth (login, cookie, CSRF, refresh, me)

Endpoint:

```
| Method | Path | Role | Body |
| POST | /auth/login | publik | `{username, password}` |
| POST | /auth/logout | login | - |
| POST | /auth/refresh-token | cookie refresh | - |
| GET | /auth/me | login | - |
| GET | /auth/profile | login | - |
| PUT | /auth/profile | login | `{name, phone?}` |
| PUT | /auth/change-password | login | `{old_password, new_password(min8), confirm_password}` |
```

Sukses `POST /auth/login` (200): set cookie `access_token`, `refresh_token` (HttpOnly), `csrf_token` (bukan HttpOnly). Body:

```
{ "success": true, "message": "Login berhasil", "data": { "user": { "id": "...", "username": "admin123",
"name": "System Administrator", "email": "admin@mcbt.local", "roles": ["admin"] } } }
```

Error umum: 401 `{"success":false,"message":"Invalid credentials"}`; 403 `{"success":false,"message":"Account is disabled"}`.

Aturan implementasi:

- Login: cari user by username ATAU email; bcrypt compare; cek `is_active`; `TouchLastLogin`; terbitkan 2 JWT (claim `uid`, `typ`, `ver`).
- Refresh: parse refresh JWT -> user masih ada & aktif & `token_version` cocok -> terbitkan pasangan baru (rotasi).
- Change password: verifikasi password lama (400 bila salah), hash bcrypt, `UpdatePassword` + naikkan `token_version` (cabut semua sesi).
- Profile GET: join users+students+classes+teachers -> `{id, username, name, email, nis, class_name, nip, phone}`.

Acceptance: curl login 200 + Set-Cookie; akses /auth/me tanpa cookie -> 401; refresh memutar token; token lama setelah reset password -> 401 "Session revoked".

B4 - Middleware & grup route

Urutan global: RequestID -> RequestLogger -> Recovery -> ErrorHandler -> CSRFProtection.

- `Authenticate`: baca cookie `access_token`, parse JWT (typ=access, ver cocok DB) -> simpan `Principal{UserID,

Username, Name, Roles}` di context.
- `CSRFProtection`: untuk POST/PUT/PATCH/DELETE, header `X-CSRF-Token` harus == cookie `csrf_token` -> else 403.
- `RequireRoles(allowed...)`: principal punya salah satu role -> else 403 "Insufficient permissions".

Grup route:

```
publik : /auth/login, /auth/refresh-token, /health
login  : /auth/me, /auth/profile, /auth/change-password, /media/:id/file
staff  (admin|teacher): bank soal, soal, ujian, section, jadwal, peserta, nilai, grading, laporan, review soal, GET siswa/kelas/mapel/tahun-ajaran
admin  : master data tulis, guru, roles, impor, reset password, ekspor siswa/guru
student: /candidate/*
```

Acceptance: akses endpoint admin dengan cookie siswa -> 403; POST tanpa CSRF header -> 403 "CSRF token missing/mismatch".

B5 - Master data

File: handler/usecase/repository untuk academic_year, class, subject, teacher, student (+impor Excel, reset password).

Endpoint (admin kecuali disebut):

```
| Method | Path | Catatan |
| GET/POST | /academic-years | list paginated + search; create `{year, semester}` dup -> 409 |
| PUT/PATCH/DELETE | /academic-years/:id, /:id/activate | activate menonaktifkan lainnya |
| GET/POST | /classes | `{academic_year_id, name}`; dup per tahun -> 409 |
| PUT/DELETE | /classes/:id | |
| GET/POST | /subjects | `{code, name}`; dup code -> 409 |
| GET/POST/PUT/DELETE | /teachers, /teachers/:id | `{username,name,email,nip?,phone?,address?}` -> buat user+role teacher, password default |
| GET | /teachers/import/template, POST /teachers/import | xlsx kolom `username,name,email,nip,phone` |
| POST | /teachers/:id/reset-password | `{new_password?}` kosong=acak10; <8 -> 422 |
| GET | /students, /students/:id | **staff** (guru read-only) |
| POST/PUT/DELETE | /students, /students/:id | admin; `{username,name,email,nis,phone?,address?,class_id?}`; dup username/email/nis -> 409 |
| GET/POST | /students/import/template, /students/import | xlsx `username,name,email,nis,phone,class_name` (class_name dicocokkan ke kelas) |
| POST | /students/:id/change-class | `{class_id}` -> 422 bila kelas tak ada |
| POST | /students/:id/reset-password | sama seperti guru |
```

Contoh sukses `POST /students` (201):

```
{ "success": true, "message": "Siswa dibuat", "data": { "id": "...", "nis": "20260001", "user": { "username": "siswa1", "name": "Siswa Satu", "email": "siswa1@x.id" }, "class": { "id": "...", "name": "VII-A" } } }
```

Error umum: 409 `{"message":"email sudah digunakan"}`; 422 `{"errors":{"class_id":"kelas tidak ditemukan"}}`.

Acceptance: impor 2 baris valid -> ImportedCount 2; baris tanpa nama -> Skipped 1.

B6 - Bank soal & soal

Endpoint (staff; mutasi bank = pemilik/admin):

```
| Method | Path | Catatan |
| GET | /question-banks | semua staff lihat; tiap baris ada `can_manage` |
| POST | /question-banks | `{subject_id, code, title, description?, academic_year_id?}`; set `created_by` = pelaku; status draft |
| POST | /question-banks/:id/clone | clone + soal; **pemilik clone = pelaku**; code `-COPY-xxxx` |
| PATCH | /question-banks/:id/publish | wajib >=1 soal -> else 422 |
| PATCH | /question-banks/:id/archive | |
| PUT/DELETE | /question-banks/:id | 403 bila bukan pemilik |
| GET | /questions?bank_id=&type=&search= | |
| POST | /question-banks/:id/questions | lihat validasi di bawah |
| GET/PUT/DELETE | /questions/:id | delete/update ditolak 409 bila `is_used` (dipakai ujian / terjawab) |
| PUT | /questions/:id/options/reorder | `{ordered_ids}` |
| PUT | /questions/:id/options/:option_id | set opsi |
| GET | /questions/:id/preview | render seperti dilihat siswa |
| GET/POST | /questions/import/template, /questions/import/validate, /questions/import/process | xlsx 10 kolom: type,content,score_weight,explanation,option_a..e,answer |
```

Validasi soal: tipe harus 5 nilai valid; PG/B-S/Multi butuh 2-5 opsi (B-S boleh auto BENAR/SALAH); Multi boleh multi-kunci; Short Answer wajib `answer_keys`; skor <=0 -> 1, >999.99 -> 999.99; esai tanpa opsi.

Contoh sukses `POST /question-banks/:id/questions` (201):

```
{ "success": true, "message": "Soal dibuat", "data": { "id": "...", "type": "MULTIPLE_CHOICE", "text": "Ibu kota Indonesia?", "score_weight": 2, "options": [ { "id": "...", "option_key": "A", "label": "A", "text": "Jakarta", "is_correct": true }, { "id": "...", "option_key": "B", "text": "Bandung", "is_correct": false } ] } }
```

Error umum: 422 `{"errors":{"options":"soal ini membutuhkan 2-5 opsi"}}`; 409 `{"message":"Soal sudah digunakan ujian. Mohon buat soal baru!"}`.

Acceptance: publish bank kosong -> 422; guru A tidak bisa PUT bank guru B -> 403.

B7 - Ujian (CRUD, settings, publish/close, scope)

Endpoint (staff):

```
| Method | Path | Catatan |
| GET | /exams?search=&status=&subject_id=&academic_year_id=&page=&limit= | guru hanya lihat `created_by` miliknya; tiap item ada `attempts_count` + `can_manage` |
| POST | /exams | `{title, description?, subject_id, academic_year_id?, question_bank_id?}` - bank opsional (konsep: bank lewat section); set `created_by` |
| GET | /exams/:id | scope: guru non-pemilik -> 403 |
| PUT | /exams/:id | update core; guru wajib bank miliknya bila mengirim bank |
| PUT | /exams/:id/settings | `{duration_minutes 1-600, max_attempts 1-10, passing_grade 0-100, randomize_*, allow_backtrack, auto_submit, show_result_immediately, negative_marking, negative_value 0-100, token_enabled, allow_discussion}`; token_enabled -> generate token 6 char; off -> token null |
| PATCH | /exams/:id/publish, /exams/:id/close | SetStatus; notfound -> 404 |
| DELETE | /exams/:id | tolak 409 bila `attempts_count > 0` |
| GET | /exams/:id/questions | **review**: seluruh soal tergroup per section (+fallback bank) lengkap opsi/kunci/pembahasan |
```

Contoh sukses `POST /exams` (201):

```
{ "success": true, "message": "Ujian dibuat", "data": { "id": "...", "title": "UAS Matematika", "status": "draft", "duration_minutes": 60, "max_attempts": 1, "passing_grade": 75, "can_manage": true, "attempts_count": 0 } }
```

Error umum: 404 `{"message":"Ujian tidak ditemukan"}`; 409 `{"message":"Ujian sudah digunakan peserta dan tidak dapat dihapus"}`; 422 `{"errors":{"duration_minutes":"durasi harus 1-600 menit"}}`.

Acceptance: guru buat ujian -> muncul di list guru, tidak muncul di list guru lain; admin lihat semua.

B8 - Section & mapping soal

Endpoint (staff, scope exam):

```
| Method | Path | Body / Catatan |
| GET/POST | /exams/:id/sections | `{name, sequence(min1)}` |
| PUT/DELETE | /sections/:id | |
| GET | /sections/:id/questions | soal termapping |
| POST | /sections/:id/questions | `{question_bank_ids: [...], total_random_questions?: int}` - ambil soal dari banyak bank, buang yang sudah termapping di exam, sampling acak bila limit diisi; return `{mapped_count, skipped}`; bank tanpa soal -> 422 |
| DELETE | /sections/:id/questions/:question_id | lepas mapping |
```

Contoh sukses `POST /sections/:id/questions` (200):

```
{ "success": true, "message": "Mapping selesai", "data": { "mapped_count": 12, "skipped": 3 } }
```

Acceptance: mapping 2 bank -> inserted = kandidat unik belum termapping; total_random 5 dari 10 kandidat -> inserted 5.

B9 - Jadwal & peserta

Endpoint (staff, scope exam):

```
| Method | Path | Catatan |
| POST/GET | /exams/:id/schedules | `{start_time, end_time, token}`; end>start -> else 422; token dup -> 409 |
| PUT/DELETE | /schedules/:id | token dup -> 409 |
```

```
| POST | /schedules/:id/generate-token | token acak 6 char |
| GET | /exams/:id/participants | daftar peserta |
| POST | /exams/:id/participants/assign-class | `{class_ids:[...]}` -> kumpulkan siswa per kelas -> Assign |
| POST | /exams/:id/participants/assign-individual | `{student_ids:[...]}`; siswa tak ada -> 422 |
| DELETE | /exams/:id/participants/:participant_id | wajib peserta milik exam tsb |
```

Contoh sukses assign (200):

```
{ "success": true, "message": "Peserta ditugaskan", "data": { "assigned": 28, "skipped": 0 } }
```

Acceptance: assign kelas berisi 5 siswa -> assigned 5; assign ulang -> skipped 5 (unique constraint).

B10 - Attempt Engine (inti ujian siswa)

Endpoint (student, semua cek ownership attempt):

```
| Method | Path | Catatan |
| POST | /candidate/exams/:exam_id/validate-token | `{token}`; gate: exam published -> peserta terdaftar -> jadwal
ada & now dalam jendela -> token cocok (bila token_enabled) -> kuota sisa (attempt aktif boleh lolos) |
| POST | /candidate/exams/:exam_id/start | gate sama; resume attempt aktif bila ada; else buat attempt:
`attempt_no=used+1`, `expires_at=now+duration` |
| GET | /candidate/attempts/:id/questions | lembar soal per section: soal+opsi (TANPA kunci) + jawaban tersimpan +
flag |
| POST | /candidate/attempts/:id/answers | `{question_id, answer_value, client_timestamp}`; upsert; soal bukan
bagian ujian -> 422; expired -> 403 |
| POST | /candidate/attempts/:id/autosave | `{items:[{question_id,value}]}`; item di luar ujian di-skip; return
jumlah tersimpan |
| POST/DELETE | /candidate/attempts/:id/questions/:qid/flag | tandai ragu-ragu |
| POST | /candidate/attempts/:id/heartbeat | `{server_time, remaining_seconds, is_expired}` - sumber waktu =
server |
| POST | /candidate/attempts/:id/submit | `{confirm_submit:true}`; idempotent; expired -> tandai expired lalu
tetap submit; `show_result_immediately` -> grade objektif seketika |
| GET | /candidate/attempts/:id/discussion | wajib submitted + `allow_discussion` (else 403); kembalikan kunci,
jawaban siswa, skor, feedback |
| POST | /candidate/attempts/:id/questions/:qid/report | `{reason}`; idempotent (existing -> return); status
pending |
| GET | /candidate/results | hasil milik siswa (skor disembunyikan bila belum publish & bukan show_immediately) |
```

Matriks skoring `gradeObjective(q, answer, negativeValue)`:

- multiple_choice / true_false: cocokkan `answer_value` dengan `options[].label` yang `is_correct` (EqualFold, trim). Benar -> `score_weight`; salah & negative>0 & jawaban tidak kosong -> `-negativeValue`.
- multiple_answer: pecah `answer_value` per koma (uppercase); benar jika set = kunci persis; skor sama aturan negatif.
- short_answer: cocokkan lowercase-trim dengan tiap baris `answer_keys`.
- essay: auto=false -> menunggu koreksi manual.

Contoh sukses `POST .../submit` (200):

```
{ "success": true, "message": "Ujian dikumpulkan", "data": { "id": "...", "status": "submitted",
"submitted_at": "2026-08-26T10:00:00Z", "score": 80.0 } }
```

Error umum: 403 `{"message":"Attempt sudah tidak aktif"}` / `{"message":"Kesempatan mengerjakan sudah habis"}` / `{"message":"Token salah"}`; 422 `{"message":"Soal bukan bagian dari ujian ini"}`.

Acceptance: start dua kali -> resume attempt sama; jawab setelah expires_at -> 403 + status expired; submit 2x -> tidak dobel finalisasi.

B11 - Pembahasan & Laporan soal

Pembahasan: lihat endpoint discussion di B10 (gating `allow_discussion` + wajib submitted).

Laporan soal (staff menangani):

```
| Method | Path | Catatan |
| GET | /question-reports?status= | admin: semua; guru: hanya laporan pada ujian `created_by` miliknya |
| PATCH | /question-reports/:id/resolve | `{status: pending|reviewing|resolved|rejected, resolution}`; guru hanya
pada ujiannya (assert attempt->exam->created_by); set `resolved_by`, `resolved_at` |
```

Contoh sukses resolve (200):

```
{ "success": true, "message": "Laporan ditangani", "data": { "id": "...", "status": "resolved", "resolution":
"Kunci diperbaiki", "resolved_at": "2026-08-26T11:00:00Z" } }
```

Error umum: 422 status tidak valid; 403 guru menangani laporan ujian orang lain; 404 laporan tak ada.

Acceptance: siswa lapor soal pada ujian tanpa bank -> muncul di list admin & guru pembuat.

B12 - Grading esai, hasil & ranking

```
| Method | Path | Catatan |
| POST | /exams/:id/calculate-grades | nilai ulang objektif semua attempt submitted (esai dilewati), total = SUM
skor dari DB -> `{attempts_graded, questions_graded}` |
| GET | /exams/:id/ungraded-essays | daftar esai belum dinilai |
| PUT | /attempts/:id/grade-essay | `{score 0..bobot, feedback?}`; bukan esai -> 422; setelah update -> hitung
ulang total attempt; return jawaban terupdate |
| GET | /exams/:id/results?class_id= | ranking: urut skor desc, `rank` mulai 1, `passed = score >= passing_grade`
|
| PATCH | /exams/:id/publish-results | `{published}` toggle |
| GET | /students/:id/results | siswa hanya miliknya (admin bebas); skor nil bila belum publish &
!show_immediately |
```

Contoh sukses `GET /exams/:id/results` (200):

```
{ "success": true, "message": "Rekap nilai", "data": [ { "rank": 1, "student_name": "Eri", "nis": "123",
"class_name": "XII IPA 1", "score": 80.0, "passing_grade": 70, "passed": true, "attempts_used": 1 } ] }
```

Acceptance: esai dinilai 15 -> total attempt berubah sesuai SUM DB; skor siswa `null` sebelum publish.

B13 - Dashboard & Ekspor

```
| Method | Path | Data |
| GET | /dashboard/admin | total_siswa, total_guru, total_bank, published_banks, total_exams, published_exams,
ongoing_exams (jadwal berjalan), total_attempts |
| GET | /dashboard/teacher | bank milik guru, soal, ujian dibuat/aktif, total siswa |
| GET | /dashboard/student | assigned/completed/passed, average_score, best_score (avg & max dari skor terbaik per
ujian) |
| GET | /exams/:id/export?format=xlsx\|pdf | stream biner; header `Content-Disposition: attachment;
filename="hasil-ujian-<slug>.<ext>"`; isi rekap+ranking |
| GET | /export/students, /export/teachers | admin; daftar siswa/guru xlsx/pdf |
```

Contoh sukses dashboard student (200):

```
{ "success": true, "data": { "assigned_exams": 5, "completed_exams": 3, "passed_exams": 2, "average_score":
82.5, "best_score": 95.0 } }
```

Error umum: 400 `{"message":"format harus xlsx atau pdf"}`; 403 non-admin ekspor siswa/guru.

B14 - Profil & Media

```
| Method | Path | Catatan |
| GET/PUT | /auth/profile | lihat B3 |
| POST | /media/upload | multipart `file`; admin+guru; batas `MAX_UPLOAD_MB`; simpan ke S3/MinIO dengan random
key; return `{id, url}` |
| GET | /media/:id/file | stream dengan content-type benar |
```

Acceptance: upload gambar 2MB -> 201 + URL bisa diakses; 12MB -> 413/422.

B15 - Unit test backend (pola wajib)

1. `internal/usecase/interfaces.go`: interface consumer-defined per repo (hanya metode yang dipakai).
2. Constructor usecase menerima interface; `router.go` tetap pass concrete repo.
3. `fakes_test.go`: fake embed interface (method tak dioverride = panic), field fungsi per perilaku, rekam panggilan.
4. Jam injeksi: `now func() time.Time` di usecase yang pakai waktu (attempt engine, candidate).
5. Kontrak repo yang harus ditiru fake: `users.FindByID/FindByIdentifier` -> `(nil, nil)` bila tak ada; repo lain -> `gorm.ErrRecordNotFound`.

Target: seluruh usecase in-scope tercakup (auth, master, bank, soal, ujian, section, attempt+skoring, grading, hasil, laporan, jadwal/peserta, dashboard, profil, ekspor, impor). Jalankan `go test ./...` - semua hijau.

---

3. FASE FRONTEND

F1 - Scaffold & tema

`npm create vite@latest frontend -- --template vue-ts`; install `tailwindcss @tailwindcss/vite vue-router pinia axios lucide-vue-next class-variance-authority`.
`style.css`: `@import "tailwindcss"` + token `:root/.dark` (primary `#3d9be9`, success `#2fbf83`) + utilitas `.bg-brand-gradient` (`#34d399->#3d9be9->#8b5cf6`), `.bg-sidebar-gradient`, `.text-brand-gradient`.
Vite proxy `/api` -> `http://localhost:9090`.

F2 - Axios & stores

`lib/axios.ts`: instance `baseURL /api/v1`, `withCredentials`.
- Request: untuk POST/PUT/PATCH/DELETE inject `X-CSRF-Token` dari cookie `csrf_token`.
- Response 401: panggil `/auth/refresh-token` sekali lalu replay request; gagal -> forceLogout.

`stores/auth.ts`: bootstrap() GET /auth/me; login/logout. `stores/ui.ts`: toast + modal error global.

F3 - AppShell

Sidebar gradient gelap (`.bg-sidebar-gradient`), logo gradasi brand; menu per role (Dashboard, Ujian Saya, Hasil Saya untuk student; Ujian/Bank Soal/Laporan untuk staff; Roles + master data untuk admin). Kartu akun bawah sidebar & chip akun di header -> klik ke `/profile`. Toggle tema + tombol logout.

F4 - Komponen UI

BaseButton (variant cva + loading), BaseInput, BaseSelect, BaseSwitch, BaseModal, BaseTable, BasePagination, BaseBadge, EmptyState, LoadingState, GlobalErrorModal, AppToasts, ImportModal (upload xlsx generik).

F5 - Halaman auth & guard

LoginPage (username+password, toast error), router guard: `requiresAuth`, `requiresAdmin`, `staff`, `requiresRole`.

F6 - Master data

Halaman AcademicYears, Classes, Subjects, Teachers, Students - pola `useCrudTable` (list+search+pagination+modal create/edit+confirm delete). Students: filter kelas, reset password (custom/acak + salin), pindah kelas, impor. Teachers: reset password + impor. Tombol ekspor Excel/PDF (useExport: blob + nama dari Content-Disposition).

F7 - Bank soal

QuestionBanksPage (grid/kartu, can_manage sembunyikan edit/hapus/publish/clone) + detail bank (BankQuestionsPage): daftar soal, form soal (tipe, konten, opsi dinamis 2-5, kunci, bobot, pembahasan), reorder opsi, impor Excel.

F8 - Ujian, Section, Review

ExamsPage: tabel + filter status/mapel + modal create/edit (TANPA bank) + publish/close + aksi per baris (review, hasil, jawaban, grading, jadwal, sections, settings, edit, hapus - sembunyikan mutasi bila `can_manage === false`). Modal settings lengkap (durasi, attempt, KKM, randomisasi, negatif, token, pembahasan).
ExamSectionsPage: kartu section + modal mapping soal (chip multi-bank + pencarian + counter) + daftar termapping (lepas).
ExamReviewPage: ringkasan total soal/skor/per jenis + filter jenis + kartu soal per section (kunci hijau, pembahasan).

F9 - Jadwal & peserta

ExamSchedulePage: form jadwal (start/end datetime + token + generate), daftar peserta, assign per kelas/individual (BaseSearchSelect siswa), hapus peserta.

F10 - Ujian siswa

CandidateExamsPage: kartu ujian (status, jadwal, tombol masuk -> modal token bila perlu).

CandidateAttemptPage: header timer (heartbeat 10-30s, auto-submit saat habis), navigasi nomor + flag, autosave 15s, modal konfirmasi submit; setelah submit -> arahkan ke hasil/pembahasan.
StudentResultsPage: daftar hasil milik sendiri. CandidateDiscussionPage: kunci hijau, jawaban siswa, pembahasan.

F11 - Grading & hasil admin

ExamResultsPage: tabel ranking (medali 3 besar, badge lulus) + filter kelas + publish hasil + ekspor Excel/PDF.
GradingPage: daftar esai belum dinilai + form nilai (0-bobot, feedback) -> simpan.
ExamAnswersPage: jawaban per siswa (sheet).

F12 - Laporan, Dashboard, Profil

QuestionReportsPage: filter status, kartu laporan (resolusi tampil), modal Tangani (wajib resolusi).
DashboardPage: kartu statistik per role + panel ekspor admin dipindah ke menu siswa/guru.
ProfilePage: info ringkas + edit nama/telepon (sembunyikan telepon utk admin) + ubah password (min 8, konfirmasi).

Acceptance global FE: semua halaman build tanpa error TS; alur login -> semua menu sesuai role berfungsi.

4. INFRASTRUKTUR

Dockerfile (backend): multi-stage `golang:1.27-alpine` build `api` + `migrate` -> runtime alpine non-root, `CMD sh -c "./migrate up && exec ./api"`.

frontend/Dockerfile: `node:20-alpine` (`npm ci && npm run build`) -> `nginx:1.27-alpine` + `nginx.conf` (try_files SPA; `location /api/ { proxy_pass http://api:8080; client_max_body_size 12m; }`).

docker-compose.yml: `postgres:16-alpine` (healthcheck pg_isready), `redis:7-alpine` (healthcheck ping), `minio` + `createbucket` (mc mb), `api` (env DB_HOST=postgres, S3_ENDPOINT=http://minio:9000, JWT_SECRET wajib), `web` (ports WEB_PORT:80). Volumes: pgdata, redisdata, miniodata.

.env.example: APP_*, DB_*, LOG_LEVEL, JWT_SECRET (wajib), JWT_TTL, COOKIE_*, S3_*, port port.

5. VERIFIKASI AKHIR

```
# backend
go build ./... && go vet ./... && go test ./...
# frontend
cd frontend && npm run build && npm test
# infra
docker compose config -q && docker compose up -d --build
```

Checklist E2E manual:

1. Login admin -> dashboard statistik tampil.
2. Buat tahun ajaran aktif -> kelas -> mapel -> guru -> siswa (impor 2 baris).
3. Guru: bank soal -> 5 soal (PG, B-S, Multi, Esai, Isian) -> publish bank.
4. Guru: ujian (tanpa bank) -> section -> mapping bank -> publish -> jadwal+token -> assign kelas.
5. Review soal (/exams/:id/review) -> kunci & pembahasan benar.
6. Siswa: login -> ujian muncul -> masuk token -> kerjakan (flag, autosave, heartbeat) -> submit.
7. Siswa: lapor 1 soal -> guru resolve -> siswa lihat pembahasan (bila aktif).
8. Guru: nilai esai -> admin: calculate grades -> publish hasil -> siswa lihat skor.
9. Admin: ekspor hasil (xlsx+pdf) & daftar siswa/guru -> file valid.
10. Reset password siswa (custom + acak) -> login dengan password baru -> token lama ditolak.

Selesai. Semua gate hijau = aplikasi setara MCBT lengkap.